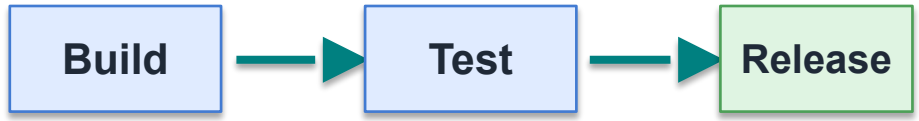
Application Deployment Strategies

Software Development Technologies

Nazih Errahel – INRTU / BRICS School

Outline

1. What is Deployment
2. Deployment Challenges
3. Deployment Strategies
4. From CI/CD to Deployment
5. Lab Preview



How do we release safely?

What is Application Deployment

Making an application available to real users.

Install

place the app

Configure

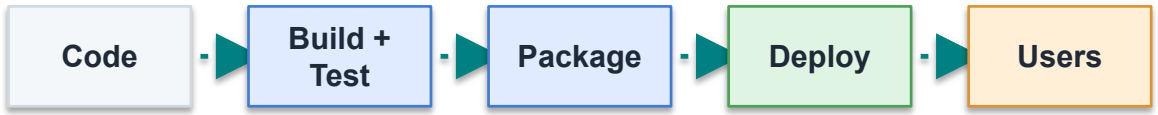
set runtime values

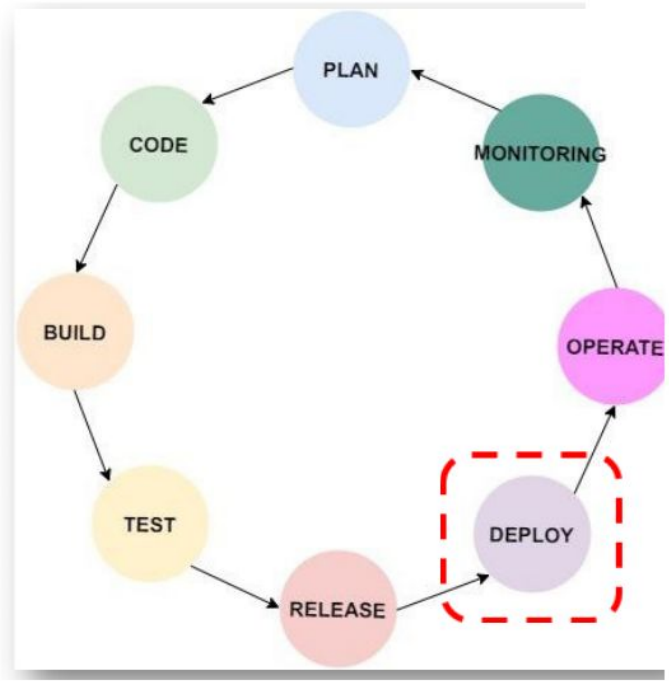
Expose

make it reachable

Deployment Phase in DevOps

- CI/CD prepares the version
- Deployment releases it
- Users must reach it





Deploy is where the release becomes available to users

Why Automate Deployment

Manual

- Slow
- Inconsistent
- Easy to forget
- Hard to repeat

Automated

- Repeatable
- Faster
- Traceable
- Rollback-ready

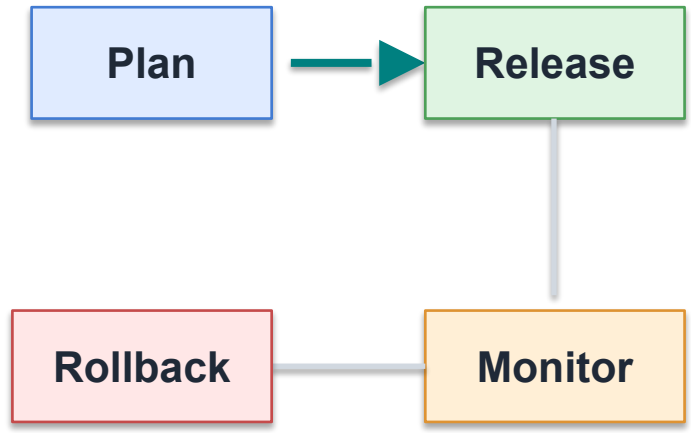
Key Deployment Challenges

Downtime	Failures	Rollback	Human errors
unavailable	bad release	return path	wrong step

Deployment strategies control these risks.

What Makes a Release Safe?

- Working version kept
- Traffic moved deliberately
- Health checked
- Rollback planned



Safe release thinking

Deployment Strategies

Blue-Green

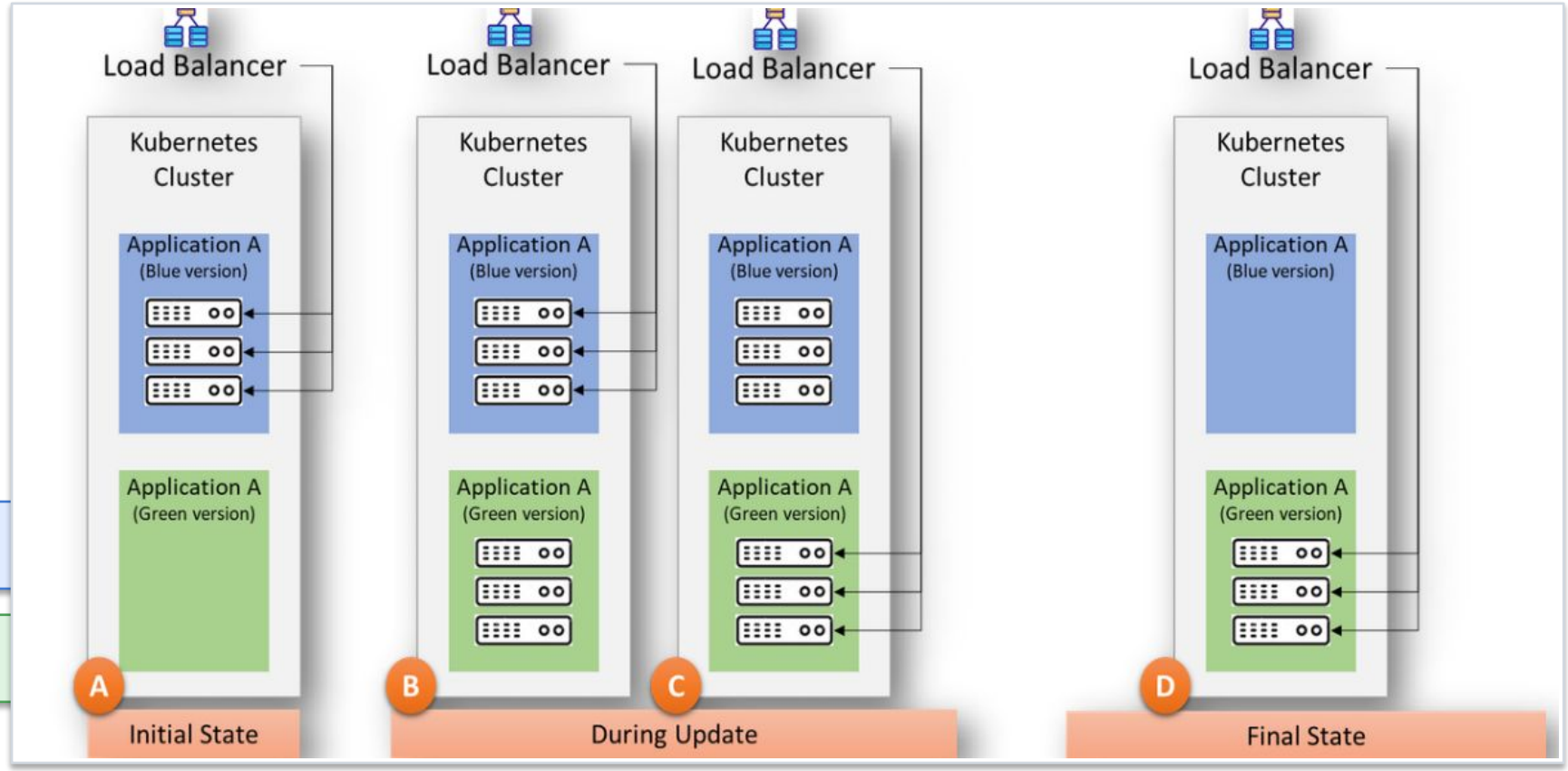
Canary

Blue-Green Deployment

- Two environments
- Switch traffic
- Instant rollback

Blue = current version

Green = new version



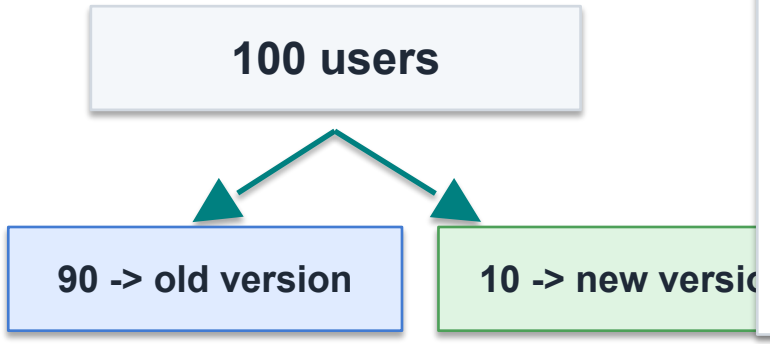
Switch traffic to Green; return to Blue if needed

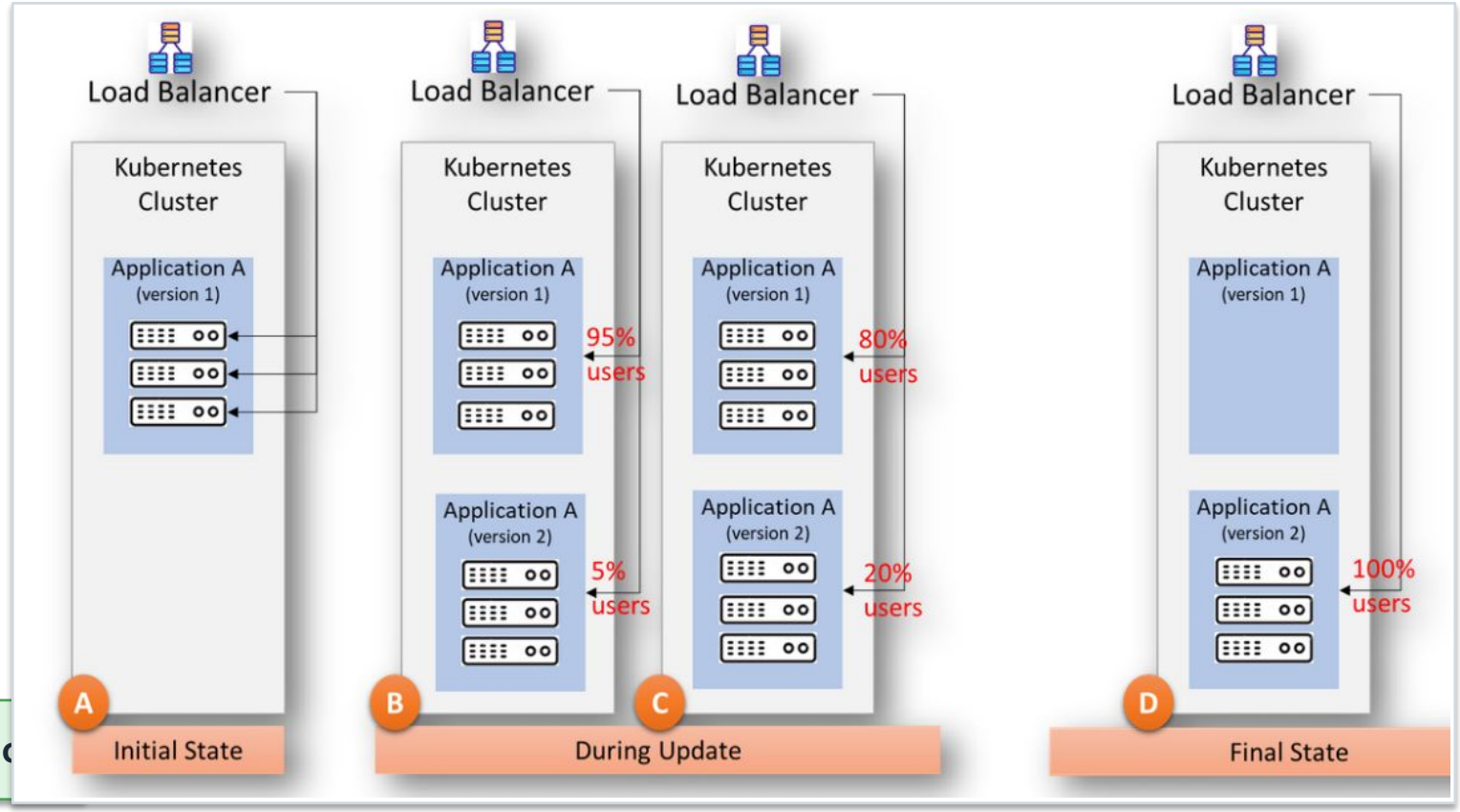
Blue-Green in Practice

- Run app_v1 + app_v2
- Change Nginx upstream
- Reload Nginx

Canary Deployment

- Small group first
- Monitor
- Increase gradually

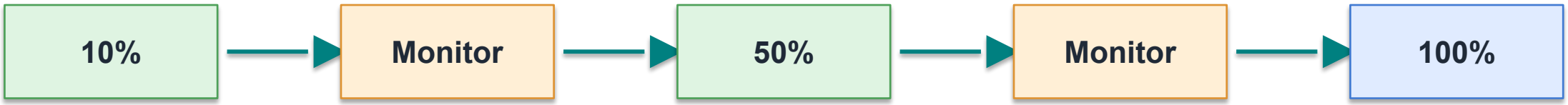




Canary in Practice

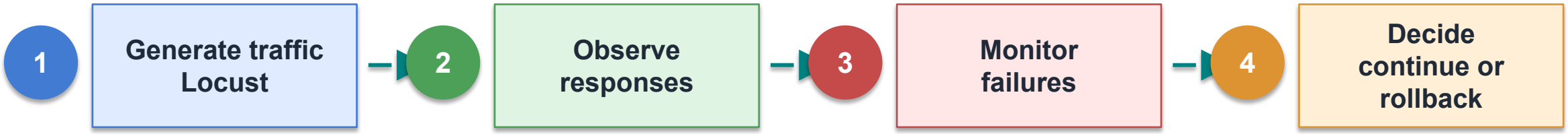
- Split traffic (e.g. 90% / 10%)
- Monitor system
- Increase gradually

The canary grows only if the new version behaves well.



Traffic can be generated using Locust

Observing Deployment Behavior



Do not release blindly: observe, then choose the next step.

Blue-Green vs Canary

Blue-Green

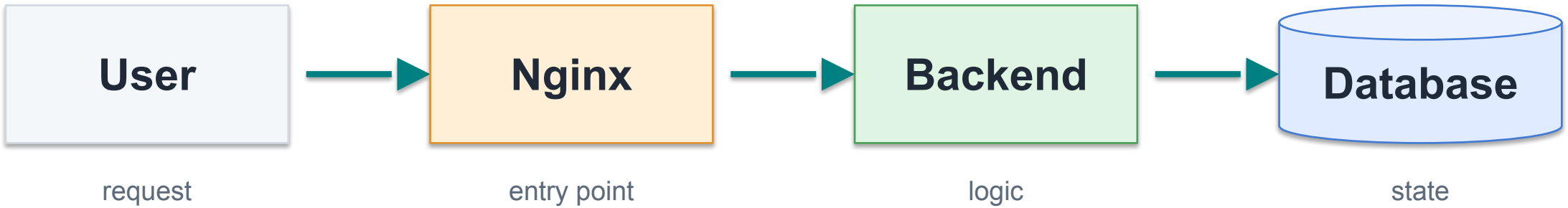
- Fast switch
- Instant rollback
- More resources

Canary

- Gradual rollout
- Lower user risk
- Slower release

Choose by risk, speed, and available resources

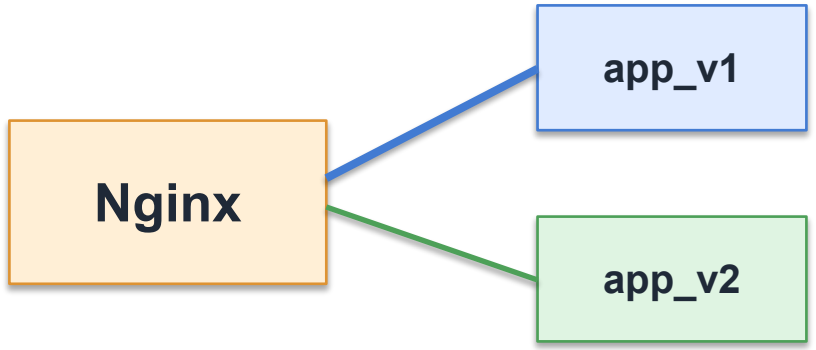
Typical Deployment Architecture



Traffic path

Nginx as Reverse Proxy

- Receives requests
- Forwards to backend
- Switches versions
- Splits traffic



Blue-Green:
one upstream active

Canary:
weighted upstreams

Docker Compose: Two App Versions

Run both versions

docker compose up -d

Nginx decides who receives traffic.

docker-compose.yml

```
services:
  app_blue:
    image: demo-app:v1
    environment:
      VERSION: "blue"

  app_green:
    image: demo-app:v2
    environment:
      VERSION: "green"

  nginx:
    image: nginx:alpine
    ports:
      - "8080:80"
```

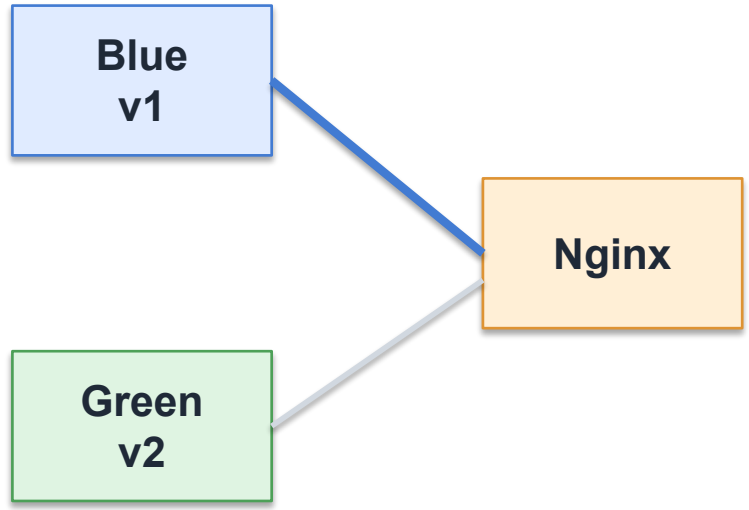
Nginx Config: Blue-Green Switch

nginx.conf

```
upstream active_backend {
    server app_blue:8080;
    # switch to app_green:8080 for release
}

server {
    listen 80;

    location / {
        proxy_pass http://active_backend;
    }
}
```



nginx -s reload

Release = change target, then reload

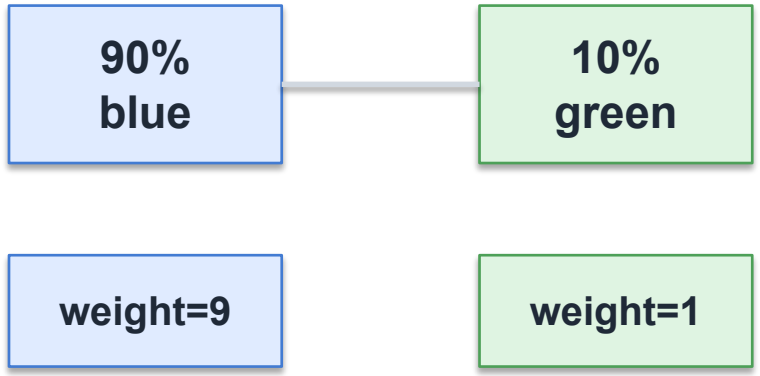
Nginx Config: Canary Split

nginx.conf

```
upstream backend {
    server app_blue:8080 weight=9;
    server app_green:8080 weight=1;
}

server {
    listen 80;

    location / {
        proxy_pass http://backend;
    }
}
```



Increase only if healthy

curl http://localhost:8080

Local Deployment Flow

Local only: Docker Compose + Nginx

Lab: What You Will Do

1. Run BLUE and GREEN app versions
2. Blue-Green: switch traffic
3. Rollback: return to previous version
4. Canary: split traffic
5. Generate traffic using Locust
6. Simulate failure in GREEN
7. Observe responses and logs
8. Decide: continue or rollback

Deployment Decision Checklist

Choose Blue-Green when...

- Instant rollback
- Two versions available
- Full traffic switch

Choose Canary when...

- Lower user risk
- Health metrics available
- Slower rollout

Always plan monitoring and rollback.

Key Takeaways

1. Deployment connects CI/CD to real users
2. Blue-Green = fast rollback
3. Canary = safer rollout
4. Monitoring is essential during deployment
5. Release decisions need evidence

Build -> Test -> Package -> Release safely

Good engineers don’t just build systems.

They release them safely.